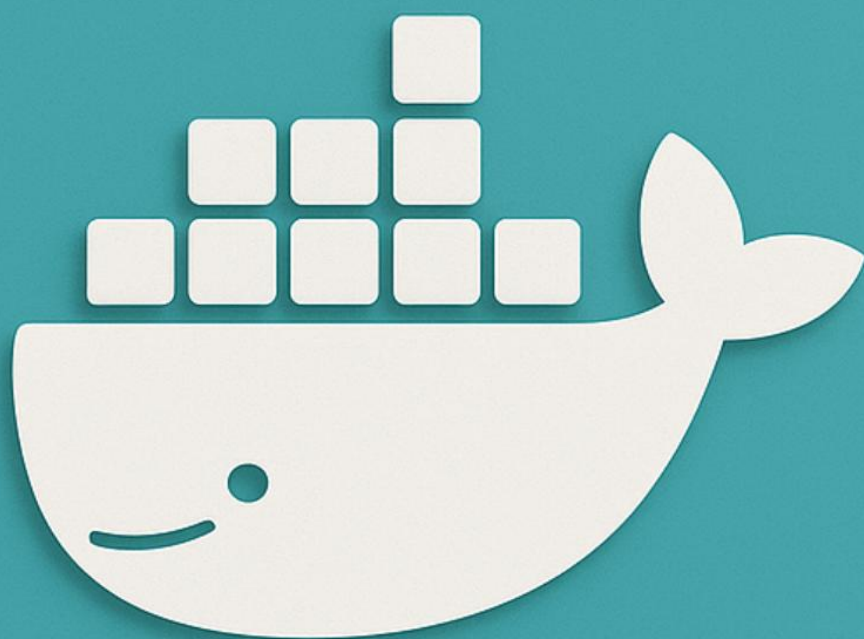


DOCKER

SCENARIO BASED IMPLEMENTATIONS



docker

By DevOps Shack

DOCKER SCENARIO BASED IMPLEMENTATIONS

DevOps Shack

1. `docker build -t myapp:1.0 -f docker/Dockerfile.prod .`

This builds an image using a specific Dockerfile (Dockerfile.prod) located in the docker folder. The tag myapp:1.0 helps in versioning.

✓ **Use Case:** You have different Dockerfiles for dev and prod environments.

2. `docker build --no-cache -t cleanbuild:v2 .`

Forces Docker to **skip using cache**, ensuring every instruction runs fresh (e.g., for package updates).

Tagging it cleanbuild:v2 helps identify this explicit rebuild.

✓ **Use Case:** Needed when you suspect stale layers are causing bugs.

3. `docker build --build-arg NODE_ENV=production -t nodeapp:prod .`

Passes a **build-time variable** (NODE_ENV) to the Dockerfile (using ARG NODE_ENV inside it). The app can use this to configure production-specific code paths.

✓ **Use Case:** Changing environment context at build time for conditional instructions.

4. `docker build -t registry.local/myorg/backend:$(git rev-parse --short HEAD) .`

Tags the image using the **current Git commit hash**, making it easy to trace the code snapshot used to build it.

✓ **Use Case:** Perfect for CI pipelines to version images per commit.

5. `docker tag myapp:latest myregistry.com/myapp:release-$(date +%Y%m%d)`

Adds a new tag to an already-built image using the **current date**, helpful for **daily snapshots**.

✓ **Use Case:** Organize image history chronologically for audits.

6. `docker run -it --rm --memory="512m" --cpus="1.5" myapp:prod`

Runs a container with **limited memory and CPU**, automatically removes it after exit.

✅ **Use Case:** Simulate how your app performs under constrained resources or enforce quotas.

7. `docker run -u 1001:1001 -it myapp bash`

Runs a container as a **non-root user (UID 1001)**. Helps enforce security practices.

✅ **Use Case:** Ensure the app doesn't require root access to function.

8. `docker run --read-only -v /tmp --tmpfs /tmp myapp`

Launches the container as **read-only**, except /tmp which remains writable (via tmpfs).

✅ **Use Case:** Lock down file system to prevent file tampering during runtime.

9. `docker run -d --restart=on-failure:3 alpine sh -c "sleep 1 && exit 1"`

`docker ps -a`

`docker inspect be9acbd3f62d | grep -i restartcount`

If the container exits with an error code, Docker will **restart it up to 3 times**.

✅ **Use Case:** For unstable apps or networked microservices that occasionally crash.

10. `docker run -d \`

`--log-driver=json-file \`

`--log-opt max-size=10m \`

`--log-opt max-file=3 \`

`--name mylogger \`

`alpine sh -c "while true; do echo Hello >> /dev/stdout; sleep 0.01; done"`

Sets the logging driver to JSON with a **size-limited log rotation**.

✅ **Use Case:** Prevent disk bloat from excessive logs in long-running containers.

11. HEALTHCHECK CMD `curl --fail http://localhost:8080/health || exit 1`

<https://github.com/jaiswaladi246/Healthcheck-docker-demo.git>

`docker build -t healthcheck-app .`

`docker run -d --name hc-demo healthcheck-app`

In a Dockerfile, this **periodically checks** if the service is alive. Non-zero exit means unhealthy.

✅ **Use Case:** Used by Docker and orchestrators (like Swarm/K8s) to restart failing containers.

12. `docker logs -f --tail 50 mycontainer`

Tails the last 50 lines and **follows the logs live**, like `tail -f`.

✅ **Use Case:** Live debug app behavior after deployment.

13. `docker logs -f --tail 50 mycontainer`

Tails the last 50 lines and **follows the logs live**, like `tail -f`.

✅ **Use Case:** Live debug app behavior after deployment.

14. `docker exec -it mycontainer sh -c "ps aux | grep java"`

Run a **custom process inspection command** inside a running container.

✅ **Use Case:** Check if your Java service is really running inside.

15. `docker run --pids-limit 100 --memory=256m --read-only myapp`

This command applies **three constraints**:

- Limits the container to **100 processes**
- Caps memory at **256 MB**
- Sets the root filesystem to **read-only**

✅ **Use Case:** Secure environments like financial or healthcare apps where process and memory abuse must be prevented.

16. `docker run -v $(pwd)/logs:/var/log/nginx nginx`

Binds the host machine's current directory logs/ to NGINX's container logs directory.

✅ **Use Case:** Developers can live-stream and inspect container logs from the host.

17. `docker-compose logs -f backend`

Tails logs for the backend service. It shows live logs (-f) from the app.

✅ **Use Case:** Debugging errors or reviewing startup behavior.

18. `docker system prune -a --volumes -f`

Aggressively removes **all unused containers, networks, volumes, and images** (even tagged ones).

✅ **Use Case:** Cleanup of development environments or CI agents after builds.

19. `docker rmi $(docker images -f "dangling=true" -q)`

Deletes all "dangling" (untagged) images that are no longer needed.

✅ **Use Case:** Reclaims disk space without affecting named images.

20. `docker rm -f $(docker ps -aq)`

Forcefully removes all containers, regardless of their state.

✅ **Use Case:** Complete reset of the Docker environment.

21. `docker image ls -f "since=ubuntu"`

Filters and lists images that were **created after the ubuntu image**.

✅ **Use Case:** Quickly find recently added custom images during a testing session.

22. `docker rename oldcontainer newcontainer`

Renames an existing container to a new name.

✅ **Use Case:** Standardize container names or avoid naming collisions in shared environments.

23. docker update --restart=always mycontainer

Changes a container's restart policy to **always** restart even after reboot.

✓ **What --restart=always means:**

- **Always** restart the container if:
 - It crashes (non-zero exit),
 - You **reboot the VM** or Docker service,

✓ **Use Case:** Ensure critical containers come back up automatically on VM restarts.

24. docker pause mycontainer && docker unpause mycontainer

Temporarily **freezes and unfreezes** the container (CPU processes are suspended).

✓ **Use Case:** Useful when performing host-level maintenance without losing container state.

25. docker diff mycontainer

Shows **file-level changes** made to the container's filesystem compared to the image it came from.

✓ **Use Case:** Inspect unexpected changes inside the container, such as temp files or runtime state.

26. docker save myapp > myapp.tar

Exports the image into a .tar archive file for **offline sharing or backup**.

✓ **Use Case:** Share image without using a Docker registry, especially in air-gapped environments.

27. docker load < myapp.tar

Loads a saved image back into the local Docker daemon.

✓ **Use Case:** Restore a saved image or move between machines that don't have internet.

28. `docker cp mycontainer:/var/log/app.log ./app.log`

Copies a file from a container to the host machine.

✅ **Use Case:** Retrieve logs or reports from a container after completion.

29. `docker run -d --name mysql-db -e MYSQL_ROOT_PASSWORD=root -v mysql-data:/var/lib/mysql mysql:8`

Runs MySQL with:

- A root password
- A volume for persistence
- In detached mode with the name mysql-db

✅ **Use Case:** Standard database setup for development or microservices.

`docker exec -it mysql-db mysql -uroot -proot`

30. `docker stats --no-stream`

Prints real-time **resource usage (CPU, MEM, NET, DISK)** per container, one-shot output.

✅ **Use Case:** Monitor containers without continuous stream (good for scripting & logs).